

Fruit Match

하진혁

hjhmoon61@naver.com

https://github.com/HaJinhyeok/2D_3Match.git

<https://youtu.be/AlsaoWX-nkc>

목차

a table of contents

1 >> 게임 개요

2 >> 게임 진행

3 >> 구조 및 설계

장르

2D, 3매치

플랫폼

PC

기술 스택

Unity6, C#, C++(IOCP)

주제

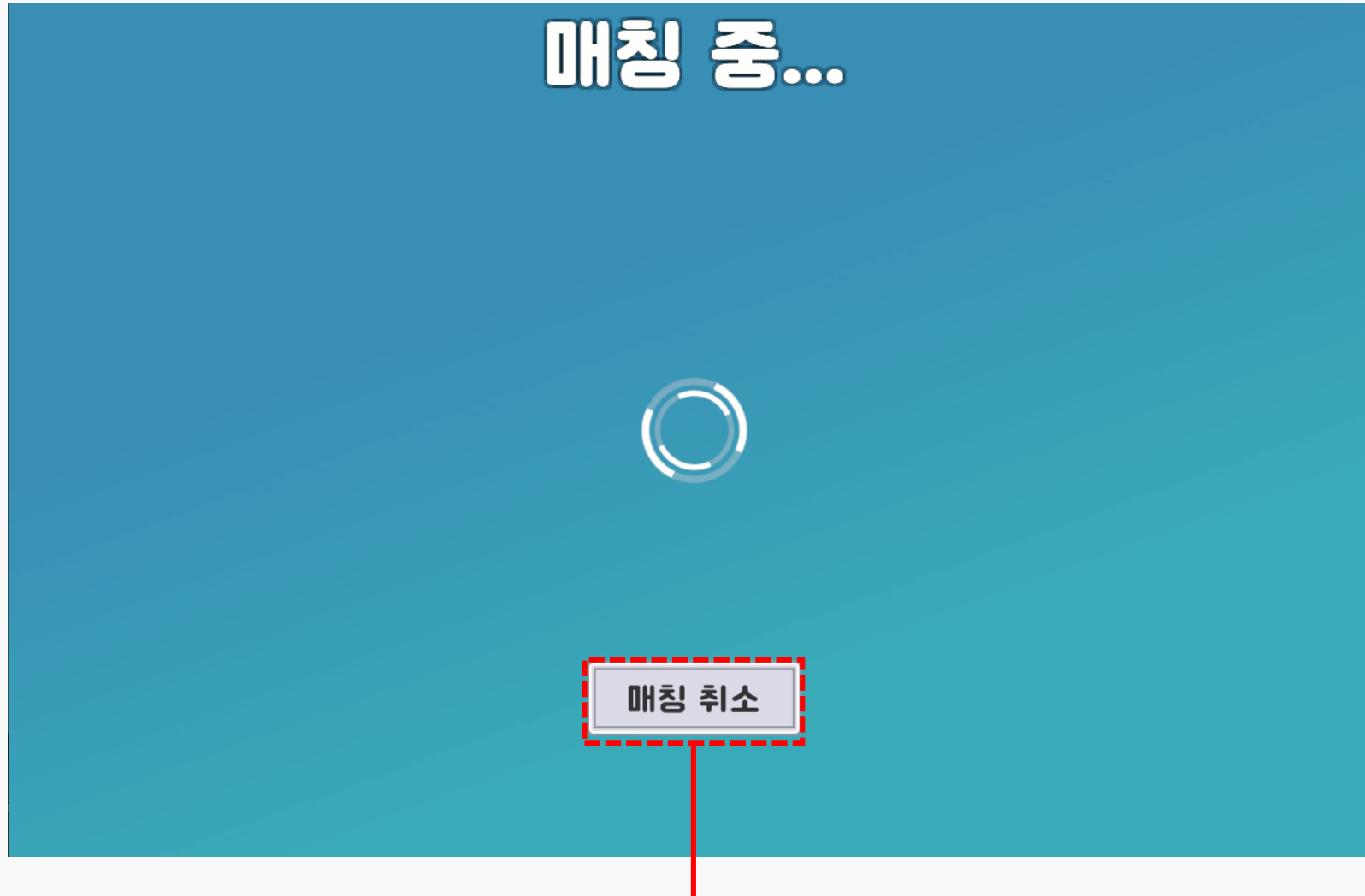
IOCP 서버를 활용한 1대1 3매치 게임



Part 2 **게임 진행** 솔로모드







매칭 취소 버튼

Part 2 **게임 진행** 1대1 매치모드

상대방
게임보드



00003

PLAYER2

상대방
점수 및 이름

플레이어
게임보드



00011

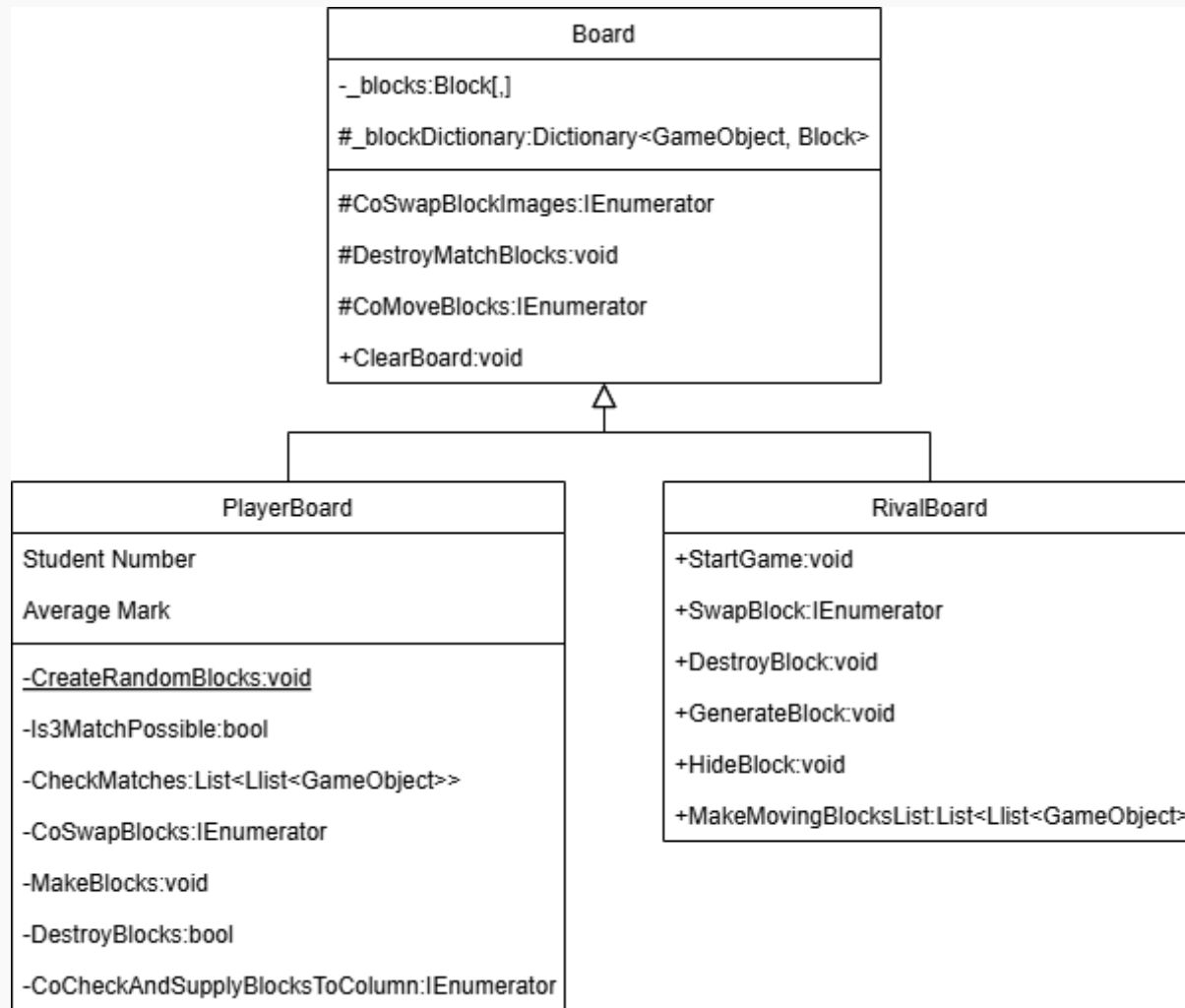
PLAYER1

Part 2 **게임 진행** 1대1 매치모드



Part 3 구조 및 설계

게임보드



Board 클래스 상속도

블록 이미지 이동, 제거 등
Player와 Rival의 공통 기능은
Board를 통해 상속

Part 3 구조 및 설계 플레어보드

```
// 3-match가 발생했는지 여부 확인
// 발생 시, 해당 블록들의 리스트들을 반환
참조 2개
public List<List<int>> CheckMatches()...

// idx번째 row 혹은 column 조사
참조 10개
List<List<int>> CheckingRowOrColumn(int idx, bool flag)...

// 블록 움직였을 때 3매치가 만들어지는지 여부 검사
// 블록 움직여서 3매치가 하나라도 만들어지면 true
// column%2==0 || row%2==0인 블록에 대해서만 실시
// 해당 블록
참조 2개
bool Is3MatchPossible()...

// first번 블록과 second번 블록을 swap했을 때 3match가 만들어지는지 검사
참조 4개
bool Is3MatchPossibleOnBlocks(int first, int second)...

// first번 블록과 second번 블록의 HintEffect 출력
참조 4개
void GiveHintOnBlocks(int x1, int y1, int x2, int y2)...
```

```
참조 1개
void OnStartDrag(GameObject go)...

참조 1개
void OnEndDrag(GameObject go)...

참조 1개
IEnumerator OnEnterBlock(GameObject go)...
```

```
string clientData = $"{(int)Define.DataStatus.Destroy}\n";
for (int i = 0; i < matchBlocks.Count; i++)
{
    if (i != 0) clientData += ' ';
    clientData += matchBlocks[i].ToString();
}
if (GameManager.s_isNetworkOn)
{
    GameManager.Client.SendMessageToServer(clientData);
}
```

마우스 입력 처리

3매치 여부 체크 등 계산 작업

힌트 제공

블록 정보 서버 전달

Part 3 구조 및 설계 상대방보드

```
List<List<GameObject>> MakeMovingBlocksList(string data)
{
    List<List<GameObject>> movingBlocks = new List<List<GameObject>>();
    string[] blockInfoLine = data.Split('\n');
    // block numbering
    int i = 0;
    // column index numbering
    int idx = 0;
    foreach (string str in blockInfoLine)
    {
        movingBlocks.Add(new List<GameObject>());
        if (string.IsNullOrEmpty(str))
        {
            idx++;
            continue;
        }
        string[] blockInfos = str.Split(',');
        foreach (string blockInfo in blockInfos)
        {
            string[] info = blockInfo.Split(' ');
            GameObject go = new GameObject("RivalMovingBlock" + i.ToString());
            i++;
            go.transform.SetParent(transform);
            go.AddComponent<Dummy>();
            RectTransform rect = go.AddComponent<RectTransform>();
            rect.sizeDelta = _size - _space;
            rect.localScale = Vector3.one;
            rect.localEulerAngles = Vector3.zero;
            rect.localPosition = new Vector3(float.Parse(info[1]), float.Parse(
                info[2]), 0);
            Image image = go.AddComponent<Image>();
            image.sprite = GameManager.Instance.BlockImages[int.Parse(info[0])];
            movingBlocks[idx].Add(go);
        }
        idx++;
    }
    return movingBlocks;
}
```

플레이어 클라이언트에서
블록 이동/제거/생성 연산 후
블록 번호 및 위치 좌표만 서버 전달



상대방 클라이언트에서는
받은 정보 기반으로 이미지만 생성

Part 3 구조 및 설계 3매치 여부 체크

```
List<List<int>> CheckingRowOrColumn(int idx, bool flag)
{
    List<List<int>> blocksOfLine = new List<List<int>>();
    List<int> blocks = new List<int>();
    Sprite currentSprite;
    // flag == true이면 idx번째 column 조사
    if (flag)
    {
        currentSprite = _blocks[0, idx].BlockImage.sprite;
        blocks.Add(idx);
        for (int i = 1; i < _numOfColumn; i++)
        {
            // 다음 블록의 이미지가 다른 거면
            if (_blocks[i, idx].BlockImage.sprite != currentSprite)
            {
                // 다음 블록의 이미지를 기준 이미지로 바꾸고
                currentSprite = _blocks[i, idx].BlockImage.sprite;
                if (blocks.Count >= 3)
                {
                    List<int> tmp = new List<int>(blocks);
                    blocksOfLine.Add(tmp);
                }
                blocks.Clear();
                blocks.Add(i * _numOfColumn + idx);
            }
            // 같은 이미지면 조사 중인 리스트에 넣고 진행
            else
            {
                blocks.Add(i * _numOfColumn + idx);
            }
        }
        if (blocks.Count >= 3)
        {
            List<int> tmp = new List<int>(blocks);
            blocksOfLine.Add(tmp);
        }
        return blocksOfLine;
    }
}
```

3매치 성립 여부는 행과 열 단위로 체크

3개 이상 연속된 동일 이미지 블록들의 번호를
리스트에 담아 반환

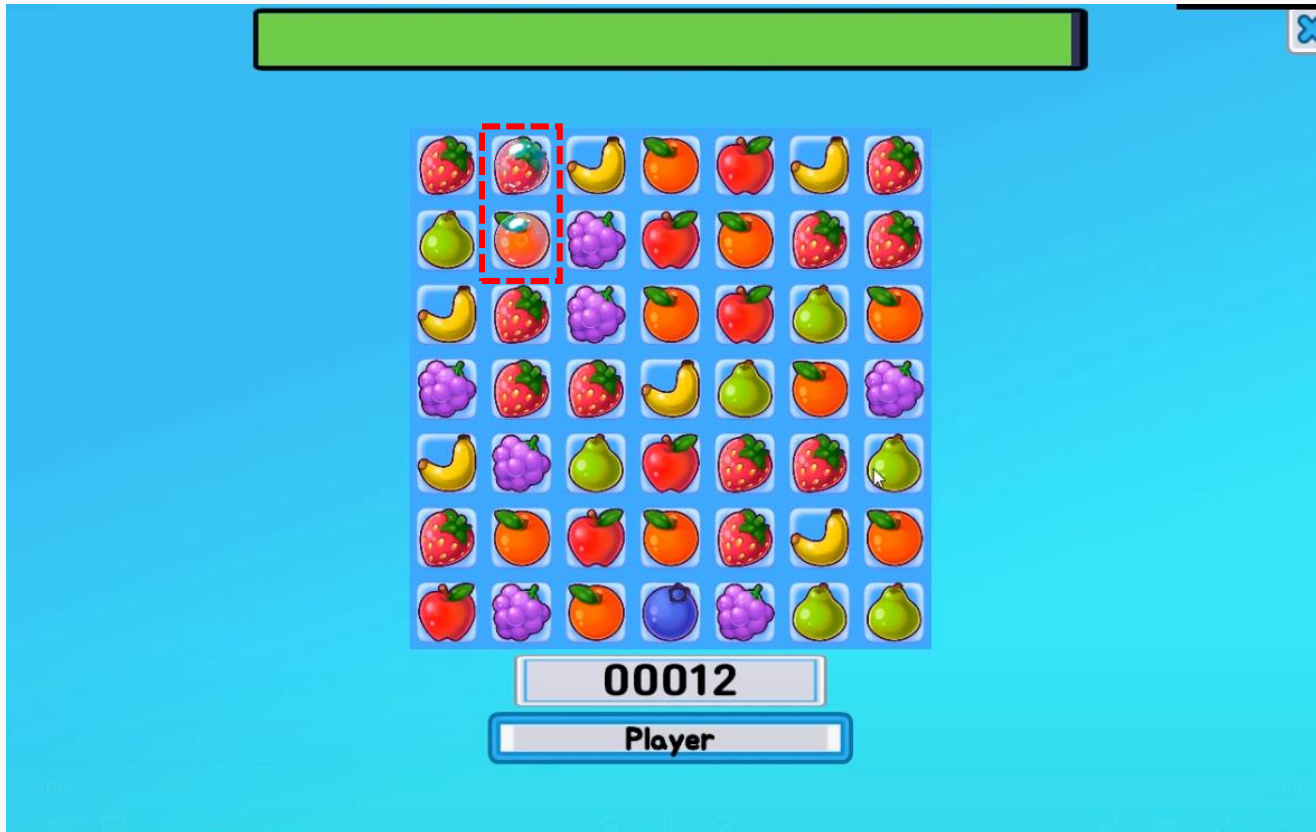
Part 3 구조 및 설계 블록제거



3매치 체크된 번호의 블록 이미지 제거

블록이 터지는 VFX 오브젝트 발생

```
protected virtual void DestroyMatchBlocks(List<int> indices)
{
    foreach (int idx in indices)
    {
        if (!_blocks[idx / _numOfColumn, idx % _numOfColumn].IsEmpty)
        {
            _blocks[idx / _numOfColumn, idx % _numOfColumn].TurnOffBlock();
            _blocks[idx / _numOfColumn, idx % _numOfColumn].BlockCrash();
            _score++;
        }
    }
    ScoreText.text = $"{_score:D5}";
}
```



```
bool Is3MatchPossibleOnBlocks(int first, int second)
{
    bool flag = false;
    int x1 = first / _numOfColumn, y1 = first % _numOfColumn;
    int x2 = second / _numOfColumn, y2 = second % _numOfColumn;
    Sprite firstImage = _blocks[x1, y1].BlockImage.sprite;
    Sprite secondImage = _blocks[x2, y2].BlockImage.sprite;
    _blocks[x1, y1].BlockImage.sprite = secondImage;
    _blocks[x2, y2].BlockImage.sprite = firstImage;

    if (CheckingRowOrColumn(x1, false).Count > 0 || CheckingRowOrColumn(x2,
        false).Count > 0
        || CheckingRowOrColumn(y1, true).Count > 0 || CheckingRowOrColumn(y2,
            true).Count > 0)
    {
        flag = true;
    }
    _blocks[x1, y1].BlockImage.sprite = firstImage;
    _blocks[x2, y2].BlockImage.sprite = secondImage;
    return flag;
}
```

일정 시간 동안 플레이어의 입력이 없을 시, 3매치 체크 후 힌트 제공

스왑 시 3매치 가능한 블록 한 쌍에 VFX 이펙트 발생

Part 3 구조 및 설계 힌트제공_3매치불가시

블록 스왑을 통해 3매치가 불가할 경우,
보드의 블록 이미지 교체



```
IEnumerator ChangeAllBlockImages()
{
    _isBlockMoving = true;
    for (int i = 0; i < _numOfColumn; i++)
    {
        for (int j = 0; j < _numOfColumn; j++)
        {
            _blocks[i, j].BlockCrash();
            _blocks[i, j].UpdateBlockImage(GameManager.Instance.BlockImages
                [UnityEngine.Random.Range(0,
                    GameManager.Instance.BlockImages.Length)]);
        }
    }
    yield return new WaitForSeconds(0.5f);
    MakeBlocks();
}
```


Part 3 구조 및 설계 블록스왑



스왑 후, 3매치 실패 시 제자리로 재스왑

Part 3 구조 및 설계 블록스왑

```
IEnumerator OnEnterBlock(GameObject go)
{
    if (MouseData.IsDragging)
    {
        Block startBlock = _blockDictionary[MouseData.StartBlock];
        Block enterBlock = _blockDictionary[go];
        Vector2 localPosDifference = enterBlock.transform.localPosition -
            startBlock.transform.localPosition;
        // 두 블록이 대각선 위치이면 이동 x
        if (Mathf.Abs(localPosDifference.x) > _size.x - _space.x && Mathf.Abs(
            localPosDifference.y) > _size.y - _space.y)
        {
            MouseData.IsDragging = false;
        }
        // 두 칸 이상 떨어진 블록 이동 불가
        else if (Mathf.Abs(localPosDifference.x) > _blockSize.x + 5 || Mathf.Abs(
            localPosDifference.y) > _blockSize.y + 5)
        {
            MouseData.IsDragging = false;
        }
    }
    else
    {
        _isBlockMoving = true;
        _isChecking = false;
        yield return StartCoroutine(CoSwapBlocks(enterBlock, startBlock));
        {
            // 만약 3매치가 없으면(==삭제할게 없으면), 두 블록 다시 제자리로
            // 교환한 두 블록에 대해서만 검사
            if (!DestroySwapBlocks(startBlock, enterBlock))
            {
                yield return StartCoroutine(CoSwapBlocks(enterBlock,
                    startBlock));
                _isBlockMoving = false;
                _isChecking = true;
            }
            else
            {
                MakeBlocks();
            }
        }
    }
}

yield return null;
```

드래그 중 다른 블록오브젝트로 진입 시 이벤트 발생

상하좌우 방향으로만 이동 가능

스왑 후 3매치가 안 되면, 제자리로 재스왑

3매치 발생 시, 해당 블록 제거 후 생성 함수 실행

Part 3 구조 및 설계 블록스왑

```
protected IEnumerator CoSwapBlockImages(Block blockA, Block blockB)
{
    Sprite blockImage1 = blockA.BlockImage.sprite;
    Sprite blockImage2 = blockB.BlockImage.sprite;

    // 각 블록 위치에 이미지 오브젝트 생성
    GameObject block1 = new GameObject("image1");
    block1.transform.SetParent(transform);
    block1.AddComponent<Dummy>();
    RectTransform rect1 = block1.AddComponent<RectTransform>();
    rect1.sizeDelta = _size - _space;
    rect1.localScale = Vector3.one;
    rect1.localEulerAngles = Vector3.zero;
    rect1.localPosition = blockA.transform.localPosition;
    Image image1 = block1.AddComponent<Image>();
    image1.sprite = blockImage1;

    GameObject block2 = new GameObject("image2");
    block2.transform.SetParent(transform);
    block2.AddComponent<Dummy>();
    RectTransform rect2 = block2.AddComponent<RectTransform>();
    rect2.sizeDelta = _size - _space;
    rect2.localScale = Vector3.one;
    rect2.localPosition = blockB.transform.localPosition;
    Image image2 = block2.AddComponent<Image>();
    image2.sprite = blockImage2;

    // 각 블록의 이미지 제거
    blockA.TurnOffBlock();
    blockB.TurnOffBlock();
}
```

```
// 서로의 위치로 이미지가 이동할 때까지 yield return null
Vector3 posA = block1.transform.localPosition;
Vector3 posB = block2.transform.localPosition;
while (Vector3.Distance(block1.transform.localPosition, posB) > 5f)
{
    blockA.TurnOffBlock();
    blockB.TurnOffBlock();
    block1.transform.localPosition = Vector3.Lerp(block1.transform.
        localPosition, posB, Time.deltaTime * 5);
    block2.transform.localPosition = Vector3.Lerp(block2.transform.
        localPosition, posA, Time.deltaTime * 5);
    yield return null;
}

Destroy(block1);
Destroy(block2);
// 끝나면 두 블록의 이미지 교체
Sprite tmp = blockImage1;
blockA.UpdateBlockImage(blockImage2);
blockB.UpdateBlockImage(tmp);
blockA.TurnOnBlock();
blockB.TurnOnBlock();
}
```

스왑할 블록 자리에 이미지 오브젝트 생성 후,
코루틴을 활용해 이미지 이동

Part 3 구조 및 설계 블록이동



이미지 오브젝트 생성 후, 코루틴을
활용해 해당 블록 자리로 이동

```
protected IEnumerator CoMoveBlocks(List<List<GameObject>> movingBlocks)
{
    // 가장 많은 블록이 새로 생성된 열을 찾아서 그 블록들이 다 제자리로 갈 때까지 옮김
    int maxColumn = 0;
    for (int i = 1; i < _numOfColumn; i++)
    {
        maxColumn = (movingBlocks[maxColumn].Count < movingBlocks[i].Count) ? i :
            maxColumn;
    }
    while (Vector3.Distance(movingBlocks[maxColumn][movingBlocks[maxColumn].Count -
        1].transform.localPosition, _blocks[0,
        maxColumn].gameObject.transform.localPosition) > 5f)
    {
        for (int j = 0; j < _numOfColumn; j++)
        {
            for (int i = 0; i < movingBlocks[j].Count; i++)
            {
                // movingBlocks[j][i] 오브젝트를
                // _blocks[movingBlocks[j].Count-1-i,j]의 localPosition으로 이동
                movingBlocks[j][i].transform.localPosition =
                    Vector3.Lerp(movingBlocks[j][i].transform.localPosition,
                        _blocks[movingBlocks[j].Count - 1 - i,
                        j].transform.localPosition,
                        Time.deltaTime * 5f);
            }
        }
        yield return null;
    }

    for (int j = 0; j < _numOfColumn; j++)
    {
        for (int i = 0; i < movingBlocks[j].Count; i++)
        {
            _blocks[movingBlocks[j].Count - 1 - i, j].UpdateBlockImage(movingBlocks
                [j][i].GetComponent<Image>().sprite);
            Destroy(movingBlocks[j][i]);
        }
    }

    movingBlocks.Clear();
}
```

Part 3 구조 및 설계 블록 생성

```
Vector3 columnTopBlockPosition;
// 빈 블록 채울 새 블록 생성해주기
for (int j = 0; j < _numOfColumn; j++)
{
    columnTopBlockPosition = _blocks[0, j].transform.localPosition;
    for (int i = 0; i < countEmpty[j]; i++)
    {
        GameObject block = new GameObject($"{movingBlocks[j].Count}_th block
            of {j}_th column");
        block.transform.SetParent(transform);
        block.AddComponent<Dummy>();
        RectTransform rect = block.AddComponent<RectTransform>();
        rect.sizeDelta = _size - _space;
        rect.localScale = Vector3.one;
        rect.localEulerAngles = Vector3.zero;
        rect.localPosition = new Vector3(columnTopBlockPosition.x,
            columnTopBlockPosition.y + (i + 1) * (_size.y + _space.y), 0);
        Image image1 = block.AddComponent<Image>();
        image1.sprite = GameManager.Instance.BlockImages
            [UnityEngine.Random.Range(0,
                GameManager.Instance.BlockImages.Length)];
        movingBlocks[j].Add(block);
    }
}
```

CoCheckAndSupplyBlocksToColumn

삭제된 블록만큼 새 블록 이미지를 생성
1대1매치 중일 경우, 서버에 생성 블록 정보 전송
생성된 블록 이동 함수 실행

```
else
{
    string clientData = $"{(int)Define.DataStatus.Generate}\n";
    foreach (List<GameObject> gameObjects in movingBlocks)
    {
        foreach (GameObject go in gameObjects)
        {
            string name = go.GetComponent<Image>().sprite.name;
            Vector2 pos = go.GetComponent<RectTransform>().localPosition;
            // 이미지 번호 - 위치 x좌표 - 위치 y좌표
            clientData += $"{name[0]} {pos.x} {pos.y}";
            // 배열의 마지막 요소를 가리키는 인덱스 표기법
            if (go != gameObjects[^1])
            {
                clientData += ',';
            }
        }
        if (gameObjects != movingBlocks[^1])
        {
            clientData += '\n';
        }
    }
    if (GameManager.s_isNetworkOn)
    {
        GameManager.Client.SendMessageToServer(hideBlockData);
        GameManager.Client.SendMessageToServer(clientData);
    }
    // 블록들 밑으로 내려주기
    yield return StartCoroutine(CoMoveBlocks(movingBlocks));
}
```

Part 3 구조 및 설계 클라이언트

```
int status = (int)msg[0] - 48;
switch (status)
{
    case (int)Define.DataStatus.Start:
        ClientReceiveProcessor.Enqueue(() =>
            GameManager.Instance.Rival.StartGame(msg.Substring(2)));

        break;

    case (int)Define.DataStatus.Swap:
        ClientReceiveProcessor.Enqueue(() =>
            StartCoroutine(GameManager.Instance.Rival.SwapBlock
                (msg.Substring(2))));

        break;

    case (int)Define.DataStatus.Destroy:
        ClientReceiveProcessor.Enqueue(() =>
            GameManager.Instance.Rival.DestroyBlock(msg.Substring
                (2)));

        break;

    case (int)Define.DataStatus.Generate:
        ClientReceiveProcessor.Enqueue(() =>
            GameManager.Instance.Rival.GenerateBlock(msg.Substring
                (2)));

        break;

    case (int)Define.DataStatus.Hide:
        ClientReceiveProcessor.Enqueue(() =>
            GameManager.Instance.Rival.HideBlock(msg.Substring(2)));

        break;

    case (int)Define.DataStatus.Result:
        ClientReceiveProcessor.Enqueue(() =>
        {
            GameManager.Instance.GameStatus.OnResultSetting(msg
                [2] - '0');
            ResultPanel.OnResultPanelOn?.Invoke();
        });

        break;
}
```

```
public class ClientReceiveProcessor : MonoBehaviour
{
    public static Queue<Action> Actions = new Queue<Action>();

    참조 12개
    public static void Enqueue(Action action)
    {
        lock (Actions)
        {
            Actions.Enqueue(action);
        }
    }

    Unity 메시지 참조 0개
    void Update()
    {
        lock(Actions)
        {
            while (Actions.Count > 0 && !RivalBoard.s_isMoving)
            {
                Actions.Dequeue().Invoke();
            }
        }
    }
}
```

서버로부터 받은 정보에 맞는 액션 수행

유니티는 메인 스레드만을 사용하므로,
별도의 액션 큐를 만들어 꺼내 쓰도록 설정

Part 3 구조 및 설계 서버

```
void CreateMatch(SOCKET client)
{
    std::lock_guard<std::mutex> lock(matchMutex);

    // 이미 1대1 매치 진행 중일 경우
    if (player1 != INVALID_SOCKET && player2 != INVALID_SOCKET)
    {
        const char* fullMsg = "MATCH_FULL";
        send(client, fullMsg, strlen(fullMsg), 0);
        printf("1:1 Match is already ongoing...\n");
        return;
    }
    // 매치 끝나고 결과에 있던 플레이어가 다음 매치 신청한 경우 or 새로운 플레이어 등록
    if (waitingPlayer == INVALID_SOCKET)
    {
        waitingPlayer = client;
        const char* waitMsg = "WAITING";
        send(client, waitMsg, strlen(waitMsg), 0);
        printf("%d is Waiting...\n", (int)client);
    }
    else
    {
        player1 = waitingPlayer;
        player2 = client;
        waitingPlayer = INVALID_SOCKET;

        const char* msg1 = "MATCHED PLAYER1 PLAYER2";
        const char* msg2 = "MATCHED PLAYER2 PLAYER1";
        send(player1, msg1, strlen(msg1), 0);
        send(player2, msg2, strlen(msg2), 0);

        IsMatching = true;
        printf("Matching Success! %d & %d\n", (int)player1, (int)player2);
    }
}
```

```
void SendMessageToPlayer(SOCKET sender, const std::vector<char>& msg)
{
    std::lock_guard<std::mutex> lock(matchMutex);

    SOCKET receiver = INVALID_SOCKET;

    // 플레이어가 나갔을 때 - 게임 종료 or 탈주
    if (msg[0] - '0' == (int)DataStatus::ExitMatch) { ... }
    // 매치 종료 시
    if (msg[0] - '0' == (int)DataStatus::Finish)
    {
        std::string str(msg.begin() + 2, msg.end());
        int score = StringToInt(str);
        if (sender == player1)
        {
            score1 = score;
        }
        else if (sender == player2)
        {
            score2 = score;
        }
        if (score1 != NO_SCORE && score2 != NO_SCORE)
        {
            char result[3];
            if (score1 > score2)
            {
                result[0] = (int)DataStatus::Result + '0';
                result[1] = '\n';
                result[2] = (int)Result::Win + '0';
                send(player1, result, sizeof(result), 0);
                result[2] = (int)Result::Lose + '0';
                send(player2, result, sizeof(result), 0);
                printf("Player1 Win\n");
            }
        }
    }
}
```

C++ 기반 IOCP 에코 서버를
변형하여 간단한 서버 구현

플레이어 출입, 점수 비교 및 결과 전달